Специальность: 5-04-0612-02 «Разработка и сопровождение программного обеспечения информационных систем»	Дисциплина: «Основы кроссплатформенного программирования»

ЛАБОРАТОРНАЯ РАБОТА № 15
Инструкционно-технологическая карта

Тема: Разработка, отладка и испытание программ с использованием системы управлениями версий Git. Работа с GitHub

Цель: Научить использовать систему управления версиями Git и платформу GitHub для разработки, отладки и испытания программ, включая создание репозитория, фиксацию изменений, работу с ветками и публикацию проекта.

Время выполнения: 4 часа

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучить теоретические сведения;
2. Ответить на контрольные вопросы;
3. Откомпилировать примеры программ из раздела «Теоретические сведения»;
4. Выполнить ИДЗ.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ДЛЯ ВЫПОЛНЕНИЯ РАБОТЫ

Система контроля версий (VCS) — это система, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определённой версии. Для контроля версий файлов в этой книге в качестве примера будет использоваться исходный код программного обеспечения, хотя на самом деле вы можете использовать контроль версий практически для любых типов файлов.

Если вы графический или web-дизайнер и хотите сохранить каждую версию изображения или макета, система контроля версий (далее VCS) — как раз то, что нужно. Она позволяет вернуть файлы к состоянию, в котором они были до изменений, вернуть проект к исходному состоянию, увидеть изменения, увидеть, кто последний менял что-то и вызвал проблему, кто поставил задачу и когда, и многое другое.

Использование VCS также значит в целом, что, если вы сломали что-то или потеряли файлы, вы спокойно можете всё исправить. В дополнение ко всему вы получите всё это без каких-либо дополнительных усилий. 1. Типы систем контроля версий

Системы контроля версий делятся на три поколения.

1.1. Локальные VCS

Многие люди в качестве метода контроля версий применяют копирование файлов в отдельный каталог (возможно даже, каталог с отметкой по времени). Данный подход очень распространён из-за его простоты, однако он невероятно сильно подвержен появлению ошибок. Можно легко забыть, в каком каталоге вы находитесь, и случайно изменить не тот файл или скопировать не те файлы, которые вы хотели.

Для того чтобы решить эту проблему, программисты давным-давно разработали локальные VCS с простой базой данных, которая хранит записи обо всех изменениях в файлах, осуществляя тем самым контроль ревизий. Пример: RCS (Revision Control System).

Недостаток: вся история хранится только на одном компьютере. Если диск выйдет из строя — вы потеряете всё.

1.2. Централизованные VCS (CVCS)

Следующая серьёзная проблема — необходимость сотрудничать с разработчиками на других системах. Для этого были созданы централизованные системы контроля версий, такие как CVS, Subversion (SVN) и Perforce. В них имеется единственный сервер, который содержит все версии файлов, а клиенты получают файлы из этого центрального хранилища.

Преимущества:

Все видят, чем занимаются остальные.

Администраторы имеют полный контроль над тем, кто и что может делать.

Недостатки:

Единая точка отказа: если сервер упадёт на час, никто не сможет сохранить изменения.

Если диск сервера повреждён и нет резервных копий — вся история проекта потеряна.

1.3. Распределённые VCS (DVCS)

Здесь в игру вступают распределённые системы контроля версий, такие как Git, Mercurial, Bazaar и Darcs. В DVCS клиент не просто получает последнюю версию файлов — он полностью копирует весь репозиторий, включая всю историю.

Преимущества:

Каждый клиент — это полноценная резервная копия.

Можно работать офлайн: коммитить, смотреть историю, создавать ветки.

Если сервер упадёт — любой клиент может восстановить проект.

2. Что такое Git?

Git — это распределённая система контроля версий, созданная Линусом Торвальдсом в 2005 году для разработки ядра Linux.

Очень важно понять эту часть материала, потому что если вы поймёте,

что такое Git и основы того, как он работает, тогда, возможно, вам будет гораздо проще его использовать. Пока вы изучаете Git, попробуйте забыть всё, что вы знаете о других системах контроля версий, таких как Subversion и

Perforce. Это позволит вам избежать определённых проблем при использовании инструмента. Git хранит и использует информацию совсем иначе по сравнению с другими системами, даже несмотря на то, что интерфейс пользователя достаточно похож, и понимание этих различий поможет вам избежать путаницы во время использования.

2.1. Ключевые особенности Git

Снимки, а не различия. Большинство VCS хранят информацию как список изменений (дельт) для каждого файла. Git хранит данные как набор снимков (snapshots) всего проекта. При каждом коммите Git делает снимок всех файлов и сохраняет ссылку на этот снимок.

Почти все операции локальны. Поскольку вся история хранится на вашем диске, большинство операций выполняется мгновенно. Целостность. Git использует контрольные суммы (SHA-1) для всего. Невозможно изменить содержимое файла или каталога, не заметив этого. Git обычно только добавляет данные. Почти любое действие в Git — это добавление данных. Сложно сделать что-то, что нельзя отменить. 3. Три состояния файлов в Git

Файлы в Git могут находиться в одном из трёх состояний:

Состояние Описание

Modified (изменён) Файл изменён, но ещё не зафиксирован в базе данных

Staged (подготовлен) Изменённый файл отмечен для включения в следующий коммит

Committed (зафиксирован) Данные безопасно сохранены в локальной базе данных

Этим состояниям соответствуют три раздела проекта Git: рабочая директория, индекс (staging area) и репозиторий.

4. Как получить репозиторий Git

Обычно вы получаете репозиторий Git одним из двух способов: Вы можете взять локальный каталог, который в настоящее время не находится под версионным контролем, и превратить его в репозиторий Git: git init

Вы можете клонировать существующий репозиторий Git из любого места:

git clone https://github.com/username/repository.git

В обоих случаях вы получите готовый к работе Git-репозиторий на вашем компьютере.

5. Основные команды Git

Команда Назначение

git init Создать новый репозиторий

git clone <url> Клонировать удаленный репозиторий

git status Показать состояние файлов
git add <файл> Добавить файл в индекс (staging)
git commit -m "сообщение" Зафиксировать изменения

git log Показать историю коммитов
git diff Показать различия между версиями
git branch Показать список веток
git checkout <ветка> Переключиться на ветку
git merge <ветка> Слить ветку в текущую
git pull Получить изменения с удаленного репозитория git
push Отправить изменения на удаленный репозиторий git
revert <коммит> Отменить указанный коммит
git reset <коммит> Сбросить состояние к указанному коммиту 6.
Ветвление в Git

Ветка — это независимая линия разработки. Ветвление позволяет:
Работать над новой функциональностью, не затрагивая стабильный код.
Экспериментировать без риска сломать основную версию.
Вести параллельную разработку несколькими командами.
Основные операции с ветками:
git branch <имя> — создать ветку
git checkout <имя> — переключиться на ветку
git checkout -b <имя> — создать и сразу переключиться
git merge <имя> — слить ветку в текущую

Конфликты слияния возникают, когда две ветки изменяют одну и ту же часть файла по-разному. Git помечает конфликтующие участки маркерами <<<<<<<, =======, >>>>>>>, и разработчик должен вручную выбрать правильный вариант.

7. Удаленные репозитории

Удаленный репозиторий — это версия вашего проекта, размещенная в интернете или в локальной сети. Основные понятия:

origin — стандартное имя для удаленного репозитория, откуда был клонирован проект.

upstream — обычно используется для обозначения оригинального репозитория при работе с форками.

Основные команды:
git remote add origin <url> — привязать удаленный репозиторий
git remote -v — показать список удаленных репозиториев
git push -u origin main — отправить изменения и запомнить связь
git fetch — получить изменения, но не сливать их
git pull — получить изменения и слить их с текущей веткой

8. Регистрация на GitHub

Первый шаг к использованию сервиса GitHub заключается в регистрации нового пользователя. В процедуре нет ничего сложного —

достаточно зайти на официальный сайт https://github.com/ и создать новую учетную запись.

Процесс регистрации

Ввод email. Система запросит рабочую электронную почту.

Создание пароля. Пароль вводится на выбор пользователя, но с учетом правил безопасности. Рекомендуется комбинация размером в 15 символов или 8, но с использованием хотя бы одной цифры и строчной буквы.

Выбор имени пользователя. Имя пользователя, как и email, проверяется на занятость, и придется выбирать тот, с которым платформа позволит продолжать регистрацию.

Настройка уведомлений. Далее нужно указать, хочется ли получать новости об обновлениях продуктов и самой системы.

Подтверждение. Последним шагом становится подтверждение — пользователю предлагается пройти капчу (в современных версиях — интерактивная проверка), после чего станет активной кнопка «Зарегистрироваться».

Подтверждение email. Вход на платформу будет открыт только после подтверждения электронной почты, поэтому зайти анонимно не получится. Это своеобразная защита сервера от многочисленных ботов и гарантия для пользователей, что они будут общаться с реальными людьми.

Современное дополнение: GitHub сейчас требует двухфакторную аутентификацию (2FA) для всех пользователей, которые вносят код. Это можно сделать через приложение-аутентификатор (Google Authenticator, Authy) или SMS.

Теперь можно приступать к управлению настройками внутри личного кабинета.

9. Работа с GitHub: Pull Request

Pull Request (PR) — это механизм, позволяющий предложить изменения в чужой репозиторий или в свою ветку.

Процесс создания PR

Форкнуть репозиторий (если это чужой проект) или создать новую ветку (если свой).

Внести изменения и запушить их.

На GitHub нажать "Compare & pull request".

Заполнить описание: что изменено и почему.

Дождаться код-ревью и ответить на комментарии.

После одобрения — "Merge pull request".

Преимущества PR

Код проходит ревью перед слиянием.

Можно обсудить изменения в комментариях.

Сохраняется история обсуждения.

10. Файл .gitignore

Файл .gitignore указывает Git, какие файлы и папки не нужно отслеживать. Это важно, потому что:

Скомпилированные файлы (.class, .jar) не должны попадать в репозиторий.

Файлы IDE (.idea/, *.iml) — это личные настройки, они не нужны другим.

Файлы с паролями и ключами не должны быть в открытом доступе.

Пример .gitignore для Java-проекта

text

```
# Compiled files
*.class
*.jar
target/

# IDE files
.idea/
*.iml
.vscode/

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
```

11. Работа с Git в IntelliJ IDEA

IntelliJ IDEA имеет встроенную интеграцию с Git и GitHub:

Действие	Как выполнить
Включить Git VCS	Enable Version Control Integration
Коммит	Ctrl+K
Push	Ctrl+Shift+K
Pull	Ctrl+T
История	Alt+9 → вкладка Log
Разрешение конфликтов	Визуальный трёхпанельный редактор
Создание PR	Git | GitHub | Create Pull Request

12. Заключение

Git — это не просто инструмент, а фундамент современной разработки. Понимание его модели (снимки, три состояния, ветки) позволяет работать эффективно и избегать типичных ошибок. GitHub, в свою очередь, превращает Git из локального инструмента в платформу для коллективной работы, код ревью и open-source разработки.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое система контроля версий и зачем она нужна? 2. Чем отличается Git от других систем контроля версий (SVN, Perforce)?

3. В каких трёх состояниях может находиться файл в Git? 4. Какие основные команды Git вы знаете?

5. Что такое ветка в Git и зачем она нужна?
6. Что такое конфликт слияния и как его разрешить?
7. Что такое GitHub и чем он отличается от Git?
8. Что такое Pull Request и зачем он нужен?
9. Что такое файл .gitignore и зачем он нужен?

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Задание. Последовательно перенести все лабораторные работы в единый Git-репозиторий, используя обычное слияние веток с созданием merge коммита.

Порядок выполнения:

− для каждой лабораторной работы создавать отдельную ветку lr1, lr2, …, lrN (или lab1, lab2, …, labN).

− по завершении работы создавать Pull Request и сливать его в основную ветку обычным merge — так, чтобы создавался отдельный коммит слияния (merge-коммит).

− перед началом следующей работы создавать новую ветку от актуального состояния основной ветки, при необходимости делать rebase (или merge) относительно master, после чего снова сливать через Pull Request.

ДОМАШНЕЕ ЗАДАНИЕ

Индивидуальное задание

Преподаватель А.С. Кибисова